Management System ISO 9001:2015
Level III Recognition for Mastery in Quality Management 2022
THE Impact Rankings 2023 +1001
Dark Green School Certified
World University Rankings Ranked 684
The WORLD UNIVERSITY RANKINGS for INNOVATION

Changing Queue Methods in Java

Name: Rodriguez, Miles J.
Date: October 21, 2026
Class Section: BSIT 2H-G1

Objective

In this activity, you will rewrite three basic Java Queue programs. You will change the methods that add, look at, and remove items to see how different Queue implementations react when they run out of items.

Problem Instructions

1. Review the three original Queue programs provided in our lesson **(LinkedList, PriorityQueue, and ArrayDeque).**
2. Rewrite each program by swapping out the methods using this guide:
   - **Instead of .offer(), use .add()**
   - **Instead of .peek(), use .element()**
   - **Instead of .poll(), use .remove()**
3. Type out, compile, and run your rewritten programs.
4. Answer the Reflective Questions at the end based on your results.

GENERAL INSTRUCTIONS
For each problem, follow these rules:

1. Import Queue Implementations: Your program must use the necessary Java utility imports:
2. Use your name in the class name: The class name must follow this format: QueueProb_FirstnameLastname
   - *For example, if the student's name is Juan Dela Cruz:* QueueProb_JuanDelaCruz.java
3. File name: The Java file must have exactly the same name as the public class name.
4. Use comments: Students should add inline comments to identify important queue operations (such as initialization, enqueueing, dequeueing, and inspecting elements).
5. Submit: For each problem, students should submit:
   - **.java source code**
   - **Screenshot of the program output**
   - **Answers to the reflective questions**

Management System ISO 9001:2015
Level III Recognition for Mastery in Quality Management 2022
THE Impact Rankings 2023 +1001
Dark Green School Certified
World University Rankings Ranked 684
The WORLD UNIVERSITY RANKINGS for INNOVATION

Expected Output

Task 1: Standard Line (LinkedList)

```
Initial Queue: [Alice, Bob, Charlie]
Next person to be served: Alice
Served: Alice
Queue after serving: [Bob, Charlie]
```

OUTPUT 1

```
Initial queue: [Alice, Bob, Charlie]
Next person to be served: Alice
Served: Alice
Queue after serving: [Bob, Charlie]
Press any key to continue . . .
```

Cleaned version

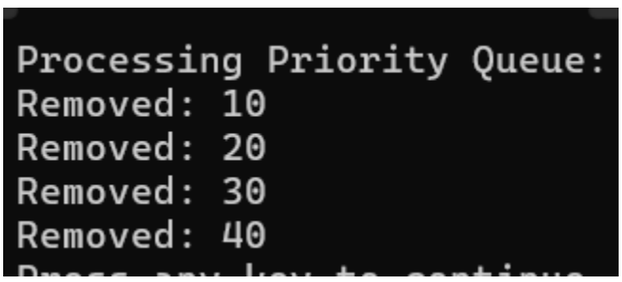
```
Initial queue: [Alice, Bob, Charlie]
Next person to be served: Alice
Served: Alice
Queue after serving: [Bob, Charlie]
Exception in thread "main" java.util.NoSuchElementException
        at java.base/java.util.LinkedList.removeFirst(Unknown Source)
        at java.base/java.util.LinkedList.remove(Unknown Source)
        at StandardFIFOQueue_Rodriguez.main(StandardFIFOQueue_Rodriguez.java:24)
Press any key to continue . . .
```

With exception encountered

Task 2: Priority Line (PriorityQueue)

```
Processing Priority Queue:
Removed: 10
Removed: 20
Removed: 30
Removed: 40
```

OUTPUT 2



Task 3: Printer Line (ArrayDeque)

```
Current Print Queue: [Tax_Return_2026.pdf, Quarterly_Report.docx, Family

Next document to print: Tax_Return_2026.pdf

--- Starting Printing Process ---
Printing: Tax_Return_2026.pdf ... Done!
Printing: Quarterly_Report.docx ... Done!
Printing: Family_Photo.jpg ... Done!

Is the print queue empty? true
```

OUTPUT 3




```
Current Print Queue: [Tax_Return_2026.pdf, Quarterly_Report.docx, Family_Photo.jpg]

Next document to print: Tax_Return_2026.pdf

--- Starting Printing Process ---
Printing: Tax_Return_2026.pdf ... Done!
Printing: Quarterly_Report.docx ... Done!
Printing: Family_Photo.jpg ... Done!

Is the print queue empty? true
Press any key to continue . . .
```

Reflective Questions

1. When you ran Task 1 (LinkedList), what specific error message or exception appeared in your console at the very end of the program?

   Task 1 encountered a NoSuchElementException as multiple remove() tried to remove an element on the queue even if the queue is already empty.

2. Why did Task 2 (PriorityQueue) and Task 3 (ArrayDeque) finish running successfully without crashing, even though they used .remove()? *(Hint: Look closely at the while loops used in those tasks).*

   Tasks 2 and 3 did not crash even if they used the .remove() method to remove an element as the program implemented a safety measure by checking first if the queue is empty or not. The program cannot proceed with removing an element if the queue is empty, implying that there is nothing to remove, preventing a crash.

3. Based on what you observed, what is the main risk of using .remove() and .element() instead of .poll() and .peek()?

   Remove() and .poll() is risky as they are affecting directly the queue by removing an element. On the other hand, element() and .peek() is considered safe and not risky as they are just used to look on a specific element on the queue.

Grading Rubric

Criteria	Excellent (5 pts)	Good (3-4 pts)	Needs Improvement (1-2 pts)
Code Correctness	All 3 programs are modified correctly with no typos.	1 or 2 programs have minor typos but the concept is understood.	Code does not compile or methods were not swapped correctly.
Execution	Student successfully ran all programs and safely triggered the Task 1 crash.	Student ran the programs but did not understand the output.	Programs were not executed.
Reflective Answers	Questions are answered thoroughly, demonstrating clear understanding of exceptions.	Answers are brief but mostly correct.	Answers are missing or completely incorrect.